02-11接口自动化Agent实战之用例数据存储

1、用例数据存储的设计

> . **关联关系：**
>
> - 接口表 ← 基础用例 ← 结构化用例
> - 基础用例是批量存储
> - 结构化用例是一条一条去存储
> - 基础用例与结构化用例是一对一关系
> - 接口与基础用例是一对多关系

1、 基础用例存储

1、表结构设计

```
CREATE TABLE base_test_case (
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键ID',
    case_id VARCHAR(100) NOT NULL UNIQUE COMMENT '用例全局唯一标识符，建议使用UUID',
    interface_id VARCHAR(100) NOT NULL COMMENT '关联的接口ID，外键关联api_interface
表',
    project VARCHAR(100) NOT NULL COMMENT '所属项目名称',
    name VARCHAR(255) NOT NULL COMMENT '测试用例名称',
    steps JSON NOT NULL COMMENT '测试步骤列表，JSON数组格式',
    expected JSON NOT NULL COMMENT '预期结果断言列表，JSON数组格式',
    dependencies JSON COMMENT '前置依赖接口列表，JSON数组格式，存储接口名称',
    status ENUM('pending', 'generated','failed') DEFAULT 'pending' COMMENT '结构
化用例状态：待生成、已生成,问题用例',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',

    -- 外键约束
    FOREIGN KEY (interface_id) REFERENCES api_interface(interface_id) ON DELETE
CASCADE,

    -- 索引
    INDEX idx_interface_id (interface_id),
    INDEX idx_project (project),
    INDEX idx_status (status)
) COMMENT='基础测试用例表，存储AI生成的基础用例信息';
```

2、完善基础用例存储

1、在State中接收接口ID

```python
# 定义一个状态节点
class StateNode(TypedDict):
    """定义一个状态管理类"""
    api_doc: str  # 接口的详细文档
    project_name: str  # 项目名称
    interface_id: str  # 接口ID，用于关联数据库中的接口
    all_api: List  # 项目下面所以的接口
    cases: Annotated[List, operator.add]  # 测试用例集
    coverage: bool  # 是否全部覆盖
    coverage_report: str  # 覆盖率报告
```

2、输出基础用例集节点保存用例

```python
def output_base_case(state: StateNode):
        """输出基础用例集并保存到数据库"""
        cases = state['cases']
        interface_id = state.get('interface_id')
        project_name = state['project_name']

        print("输出基础用例集并保存到数据库:")

        # 批量保存基础用例到数据库
        with SQLDB() as db:
            for case in cases:
                try:
                    # 生成用例唯一ID
                    case_id = str(uuid.uuid4())

                    # 准备插入数据
                    sql = """
                    INSERT INTO base_test_case (
                        case_id, interface_id, project, name, steps, expected,
dependencies, status
                    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
                    """
                    # 执行插入
                    db.cursor.execute(sql, (
                        case_id,
                        interface_id,
                        project_name,
                        case.get('name', ''),
                        json.dumps(case.get('steps', []), ensure_ascii=False),
                        json.dumps(case.get('expected', []),
ensure_ascii=False),
                        json.dumps(case.get('dependencies', []),
ensure_ascii=False),
                        'pending'
                    ))
                    print(f"基础用例已保存: {case.get('name', '')}")
                except Exception as e:
                    print(f"保存基础用例失败: {case.get('name', '')}, 错误: {e}")

        print("基础用例集保存完成")
        return {"cases": cases}
```

2、 结构化用例存储

1、表结构设计

```sql
CREATE TABLE structured_test_case (
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键ID',
    case_id VARCHAR(100) NOT NULL UNIQUE COMMENT '结构化用例全局唯一标识符',
    base_case_id VARCHAR(100) NOT NULL COMMENT '关联的基础用例ID，外键关联base_test_case表',
    interface_id VARCHAR(100) NOT NULL COMMENT '关联的接口ID，外键关联api_interface表',
    name VARCHAR(255) NOT NULL COMMENT '用例名称',
    description TEXT COMMENT '用例描述',
    project VARCHAR(100) NOT NULL COMMENT '所属项目名称',
    module VARCHAR(100) COMMENT '所属模块名称',
    interface_name VARCHAR(255) COMMENT '接口名称',

    -- 前置条件信息
    preconditions JSON COMMENT '前置条件列表，包含前置步骤的完整请求信息',

    -- 主请求信息
    request JSON NOT NULL COMMENT '主请求信息，包含method、url、headers、params、body等',

    -- 断言信息
    assertions JSON NOT NULL COMMENT '断言信息，包含响应断言和数据库断言',

    -- 用例状态和审核信息
    review_status ENUM('pending', '通过', '失败', '错误') DEFAULT 'pending' COMMENT '用例审核状态',
    test_report TEXT COMMENT '测试报告，记录用例执行结果和错误信息',
    generation_count INT DEFAULT 1 COMMENT '用例生成次数，用于重新生成计数',

    -- 时间字段
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',

    -- 外键约束
    FOREIGN KEY (base_case_id) REFERENCES base_test_case(case_id) ON DELETE CASCADE,
    FOREIGN KEY (interface_id) REFERENCES api_interface(interface_id) ON DELETE CASCADE,

    -- 索引
    INDEX idx_base_case_id (base_case_id),
    INDEX idx_interface_id (interface_id),
    INDEX idx_project (project),
    INDEX idx_review_status (review_status)
) COMMENT='结构化测试用例表，存储详细的可执行测试用例';
```

2、完善结构化用例存储

1、在State中定义接口ID和和基础用例ID

```python
class StateNode(TypedDict):
```

```
    """定义节点的状态数据"""
    project: str  # 项目名称
    base_case_id: str  # 关联的基础用例ID
    interface_id: str  # 关联的接口ID
    case_api: dict  # 用例接口
    case_info: dict  # 基础用例信息
    additional_info: str  # 额外的说明信息
    other_api: list  # 前置条件依赖的接口详情
    test_data: str  # 测试数据(放在测试环境里面)
    files_list: list  # 文件上传的用例可用的文件列表
    function_list: list  # 生成测试数据和加密处理的可用的函数列表
    output_case: dict  # 最终输出的用例
    test_report: dict  # 验证用例可用性得到的测试报告
    is_available: bool  # 用例是否可用
    generator_count: int  # 用例生成的次数
    review_state: str  # 用例预执行的评审结果
```

2、保存用例节点完善保存逻辑

```
   @staticmethod
    def save_case(state: StateNode):
        """保存用例到数据库"""
        case = state.get("output_case")
        base_case_id = state.get('base_case_id')
        interface_id = state.get('interface_id')
        project = state.get('project')

        print("用例生成流程执行完毕，开始保存用例到数据库")

        try:
            with SQLDB() as db:
                # 生成结构化用例唯一ID
                case_id = str(uuid.uuid4())

                # 获取用例审核状态，判断是否为问题用例
                review_status = case.get('review_status', 'pending') if case
else '错误'
                is_problem_case = review_status in ['失败', '错误']

                # 1. 插入结构化用例数据
                sql_insert = """
                INSERT INTO structured_test_case (
                    case_id, base_case_id, interface_id, name, description,
project, module,
                    interface_name, preconditions, request, assertions,
review_status,
                    test_report, generation_count
                ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
%s)
                """

                # 执行插入结构化用例
                db.cursor.execute(sql_insert, (
                    case_id,
                    base_case_id,
                    interface_id,
                    case.get('name', '') if case else '生成失败的用例',
```

```
                    case.get('description', '') if case else '用例生成失败',
                    project,
                    case.get('module', '') if case else '',
                    case.get('interface', '') if case else '',
                    json.dumps(case.get('preconditions', []),
ensure_ascii=False) if case else '[]',
                    json.dumps(case.get('request', {}), ensure_ascii=False) if
case else '{}',
                    json.dumps(case.get('assertions', {}), ensure_ascii=False)
if case else '{}',
                    review_status,
                    state.get('test_report', ''),
                    state.get('generator_count', 1)
                ))

                # 2. 根据用例状态更新基础用例状态
                if is_problem_case:
                    # 问题用例：更新基础用例状态为失败
                    base_case_status = 'failed'
                    print(f"问题用例已保存到数据库: {case.get('name', '') if case
else '生成失败的用例'}")
                else:
                    # 正常用例：更新基础用例状态为已生成
                    base_case_status = 'generated'
                    print(f"结构化用例已保存到数据库: {case.get('name', '')}")

                sql_update = """
                UPDATE base_test_case
                SET status = %s, updated_at = CURRENT_TIMESTAMP
                WHERE case_id = %s
                """

                # 执行更新基础用例状态
                db.cursor.execute(sql_update, (base_case_status, base_case_id))

                print(f"基础用例状态已更新为{base_case_status}: {base_case_id}")

        except Exception as e:
            print(f"保存结构化用例失败: {e}")

        return {"output_case": case}
```

2、基础用例---->完整用例生成

```
# @Author  : 木森
# @weixin: python771
"""
从api文档到生成的全流程
"""
from typing import TypedDict

from langgraph.graph import StateGraph

from langgraph.constants import START, END
```

```python
from workflows.all_case_strutured_graph import StructuredCaseGeneratorFlow
from workflows.base_case_generator import BaseCaseGeneratorFlow


class StateNode(TypedDict):
    """工作流的状态管理"""
    # 用户输出的数据
    api_doc: dict
    project_name: str
    interface_id: str

    test_data: dict  # 测试环境中的测试数据

    # 节点数据
    base_cases: list  # 基础用例数据
    structured_cases: list  # 结构化用例数据


class CompleteCaseGenerationFlow:
    """
    case生成的全流程工作流
    """

    def __init__(self):
        #  结构化case工作流
        self.structured_flow = StructuredCaseGeneratorFlow().create_workflow()
        #   基础case生成的工作流
        self.base_flow = BaseCaseGeneratorFlow().create_workflow()

    def generate_base_cases(self, state: StateNode):
        """生成基础的用例"""
        response_state = self.base_flow.invoke(state)
        # 获取生产的基础用例
        base_cases = response_state.get('cases')
        # 获取覆盖率分析报告
        return {'base_cases': base_cases}

    def generate_structured_cases_batch(self, state: StateNode):
        """批量生成结构化用例"""
        response_state = self.structured_flow.invoke({
            "base_cases": state.get('base_cases', []),
            "api_info": state.get('api_doc'),
            "project_name": state.get('project_name'),
            "test_data": state.get('test_data', {}),
        })
        return {'structured_cases': response_state.get('structured_cases'), }

    @staticmethod
    def get_generator_report(state: StateNode):
        """获取生成的完整报告"""

    def create_workflow(self):
        """
        创建工作流
        :return:
        """
        graph = StateGraph(StateNode)
        graph.add_node('基础用例生成', self.generate_base_cases)
```

```
        graph.add_node('结构化用例生成', self.generate_structured_cases_batch)
        graph.add_node('生成报告', self.get_generator_report)
        graph.add_edge(START, '基础用例生成')
        graph.add_edge('基础用例生成', '结构化用例生成')
        graph.add_edge('结构化用例生成', '生成报告')
        graph.add_edge('生成报告', END)
        return graph.compile()


if __name__ == '__main__':
    flow = CompleteCaseGenerationFlow().create_workflow()
    api_info = {'version': None, 'description': '创建项目', 'path':
'/api/pro/projects/', 'tag': None,
                'responses': '{"201": {"id": {"type": "integer", "description":
"项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id":
{"type": "integer", "description": "项目负责人"}, "create_time": {"type":
"string", "description": "创建时间"}}, "422": {"detail": {"type": "array",
"description": ""}}}',
                'summary': '创建项目', 'method': 'POST', 'module': None,
                'requestBody': '{"name": {"type": "string", "description": "项目
名称"}, "user": {"type": "integer", "description": "项目负责人"}}',
                'project': 'web测试平台', 'parameters': '[]', 'distance':
-3.5762786865234375e-07}

    result = flow.invoke({"api_doc": api_info,
                          'project_name': "web测试平台",
                          "interface_id": "ea4a5cd5-e75c-3b69-aec7-bee41a10e189"
                          })
    print("=======生成完毕============")

    print(result)
```

- **今天上课的最后用例未存储的原因，少写了个Not**

```
    """保存用例:对当前进行进行存储，保存到mysql数据库"""
    case = state.get("output_case")   case: {'name': '创建项目-no
    if not state.get('base_case_id', ''):
        print("未传递基础用例ID，无需存储结构化用例")
        return
        # 将生成的用例保存的结构化用例表，修改基础用例的用例状态为已生成结
```